

# AGENCYOS

## PROJECT PLANNING AGENT

Phase 2 Responsibilities, Operational Blueprint, Implementation Contract & Master Checklist

APPROVED SCOPE → OPERATIONAL PROJECT BLUEPRINT → READINESS FOR EXECUTION

|                   |                                                                                    |
|-------------------|------------------------------------------------------------------------------------|
| Status            | LOCKED PHASE 2 RESPONSIBILITY BASELINE                                             |
| Role              | Project Planning Agent                                                             |
| Phase             | Phase 2 - Client Onboarding + Payment + Official Project Kick-off                  |
| Primary Input     | Accepted quotation + approved scope + inherited requirements + onboarding context  |
| Primary Output    | Versioned operational project blueprint                                            |
| Critical Boundary | NOT the Phase 5 Development Planning Agent; no code/DB/API implementation planning |

This document defines the Project Planning Agent deeply enough for direct implementation and later Claude Code execution without confusing business/operational planning with technical development planning.

# 1. Mission

The Project Planning Agent converts the already-approved commercial/project context into a structured operational blueprint that AgencyOS can execute phase by phase. It organizes what must be delivered, in what sequence, what depends on what, what the client must provide, what internal teams/agents must prepare, and what conditions make the project ready to move forward.

Locked distinction: this agent plans the project lifecycle operationally. It does NOT decide how developers should code a feature, design database tables, implement APIs, choose algorithms, or create the Phase 5 master development checklist. Those belong to the later Development Planning Agent.

## 2. Trigger & Start Conditions

- ☐ Valid Phase 2 project exists and is linked to the Phase 1 WON handoff.
- ☐ Accepted quotation and approved scope are available.
- ☐ Required inherited requirements/context are accessible.
- ☐ Required Phase 2 financial gate is verified, unless an explicitly governed exception allows planning earlier.
- ☐ PM/Orchestrator triggers the Project Planning Agent.
- ☐ Duplicate planning trigger for the same project/version is protected by idempotency.

## 3. Inputs

- ☐ Accepted quotation and commercial scope.
- ☐ Project type and agreed deliverables.
- ☐ Approved requirements gathered in Phase 1.
- ☐ Relevant client discussions, meeting notes and decisions.
- ☐ Known timelines or commitments that were explicitly approved.
- ☐ Known client assets, access and dependencies.
- ☐ Known third-party services/integrations from approved scope.
- ☐ Onboarding information collected by PM.
- ☐ Billing/payment milestone context required for operational sequencing.
- ☐ AgencyOS locked phase definitions and readiness gates.

## 4. Primary Responsibilities

### 4.1 Scope-to-deliverables conversion

- ☐ Read the accepted scope without silently adding features.
- ☐ Create a structured deliverables register.
- ☐ Map each deliverable to the relevant AgencyOS phase/workstream.
- ☐ Preserve quotation/scope references for traceability.
- ☐ Flag ambiguity instead of guessing.

### 4.2 Operational phase sequencing

- ☐ Map the project through the locked AgencyOS lifecycle.
- ☐ Identify which phases/tasks are applicable or not applicable for the project type.
- ☐ Define business-level ordering and dependencies.
- ☐ Do not change locked AgencyOS phase rules unless an authorized policy/decision explicitly permits it.

### 4.3 Timeline and milestone shell

- ☐ Create a realistic operational timeline shell using known approved commitments.
- ☐ Represent target start/end windows or durations without inventing guarantees.
- ☐ Map finance milestones and major approval points to the operational sequence.
- ☐ Record assumptions that affect the timeline.

### 4.4 Client dependency identification

- ☐ Identify information/access/assets likely required from the client before later work can proceed.
- ☐ Examples may include brand assets, content, domain/hosting, app-store access, payment/SMS/email/maps/social-login/cloud credentials or other scope-specific dependencies.
- ☐ Mark each dependency with needed-by phase/date/window, owner and status.
- ☐ Ask PM to collect client-side items; Planning Agent should not take over client communication.

### 4.5 Internal dependency identification

- ☐ Identify internal prerequisites between phases, agents and approvals.
- ☐ Map which outputs from one phase become inputs to another.
- ☐ Record blockers and critical dependencies explicitly.

### 4.6 Deliverable readiness criteria

- ☐ Define what evidence/output must exist before an operational milestone can be considered ready.
- ☐ Use objective states/evidence where possible.
- ☐ Do not equate 'agent says done' with completion.

### 4.7 Risk and blocker seed

- ☐ Create initial operational risk/blocker register.
- ☐ Record scope ambiguity, missing client dependency, external-provider dependency, approval dependency, timeline dependency and other project-specific risks.
- ☐ Assign owner/escalation path where known.

### 4.8 Planning versioning

- ☐ Persist the plan as a versioned artifact.
- ☐ Track why a version changed and which approved source caused the change.
- ☐ Do not silently rewrite historical plans.

## 5. What the Project Planning Agent Must NOT Do

- ☐ Must not write production code.
- ☐ Must not create detailed coding tasks for developers.
- ☐ Must not design database schemas/tables.
- ☐ Must not design API endpoints/contracts as a technical architect.
- ☐ Must not choose frameworks/libraries/models as an implementation decision unless already locked as project context.
- ☐ Must not perform UI design.
- ☐ Must not perform QA testing.
- ☐ Must not communicate directly with client when PM owns that interaction.
- ☐ Must not modify approved scope.
- ☐ Must not convert an ambiguous requirement into an invented requirement.
- ☐ Must not promise dates that are not supported by approved commitments/planning assumptions.
- ☐ Must not approve payment or bypass Finance/Admin gates.
- ☐ Must not replace the Phase 5 Development Planning Agent.

## 6. Project Planning vs Development Planning

| Area                   | Project Planning Agent - Phase 2                          | Development Planning Agent - Phase 5                                    |
|------------------------|-----------------------------------------------------------|-------------------------------------------------------------------------|
| Purpose                | Organize the project operationally                        | Plan detailed product implementation                                    |
| Scope                  | Deliverables, phases, milestones, dependencies, readiness | Features/modules, frontend/backend, DB, APIs, auth, integrations, tests |
| Client dependencies    | Identify early and assign/request via PM                  | Refine exact technical credentials/access needed                        |
| Timeline               | Operational timeline shell                                | Detailed implementation sequence/checklist                              |
| Technical architecture | No                                                        | Yes, where required                                                     |
| Coding task breakdown  | No                                                        | Yes                                                                     |
| Acceptance/readiness   | Business/phase readiness criteria                         | Technical acceptance/completion criteria                                |
| Output                 | Operational Project Blueprint                             | Detailed Development Implementation Plan + Master Checklist             |

Both agents are necessary because planning the agency lifecycle and planning the software implementation are different responsibilities.

## 7. Required Operational Project Blueprint

- ☐ Project identity and source references.
- ☐ Approved project objective/summary.
- ☐ Approved scope snapshot/reference.
- ☐ Deliverables register.
- ☐ Applicable AgencyOS phases.
- ☐ Operational phase sequence.
- ☐ Major milestone map.
- ☐ Finance milestone map where relevant.
- ☐ Client approval points.
- ☐ Client dependency register.
- ☐ Internal dependency register.
- ☐ Third-party/external dependency register.
- ☐ Timeline/duration shell.
- ☐ Risk/blocker register.
- ☐ Assumptions register.
- ☐ Open clarification register.
- ☐ Readiness criteria per relevant phase/milestone.
- ☐ Evidence/output expectations.
- ☐ Owners/responsible roles.
- ☐ Plan version, status and generation timestamp.

## 8. Deliverables Register

| Field              | Purpose                                    |
|--------------------|--------------------------------------------|
| deliverable_id     | Stable identifier                          |
| name               | Client/agreed deliverable name             |
| scope_reference    | Quotation/requirement source               |
| applicable_phase   | AgencyOS phase that produces/validates it  |
| owner_role         | Responsible agent/team role                |
| dependencies       | Required prior outputs/items               |
| client_dependency  | Any client-provided prerequisite           |
| readiness_criteria | What makes it operationally ready          |
| evidence_required  | Required artifact/approval/reference       |
| status             | PLANNED / BLOCKED / READY / COMPLETE / N/A |

## 9. Dependency Model

Dependencies should be explicit objects, not buried in prose.

| Dependency type    | Examples                                          | Owner/action                     |
|--------------------|---------------------------------------------------|----------------------------------|
| Client information | Business info, content, references                | PM requests from client          |
| Client access      | Domain, hosting, app stores, cloud/service access | PM coordinates secure collection |
| External service   | Payment, SMS, email, maps, storage                | Track provider/client readiness  |

| Dependency type | Examples                                  | Owner/action                   |
|-----------------|-------------------------------------------|--------------------------------|
| Internal output | Approved theme, prototype, build, QA pass | Wait for producing phase/agent |
| Human approval  | Admin/client confirmation                 | Track approval gate            |
| Finance         | M1/M2/M3/M4 payment gate                  | Finance/Admin workflow         |

# 10. Clarification Workflow

PLANNING AMBIGUITY → create ClarificationRequired → PM receives concise question/context → PM asks client → client response → PM structures response → ClarificationResolved → Planning Agent updates new plan version

- ☐ Planning Agent never guesses an unclear client requirement.
- ☐ If clarification implies a new/out-of-scope requirement, route to the appropriate scope/change process instead of silently adding it.

# 11. Timeline Planning Rules

- ☐ Use approved timeline commitments when they exist.
- ☐ Represent uncertainty as estimates/windows/assumptions rather than false certainty.
- ☐ Account for phase dependencies and approval/payment gates.
- ☐ Account for client dependencies that can block later work.
- ☐ Identify tasks that can proceed independently while another dependency is waiting.
- ☐ Do not compress the timeline merely to satisfy a requested date without evidence.
- ☐ Changes to the timeline must be versioned and attributable.

# 12. Readiness Gate Design

| Gate                      | Examples of evidence/condition                                                                              |
|---------------------------|-------------------------------------------------------------------------------------------------------------|
| Phase 2 kickoff readiness | Required onboarding sufficiently complete; WhatsApp mapped if required; M1 verified; operational plan ready |
| Phase 3 readiness         | Phase 2 complete + required approved context handed forward                                                 |
| Later phase readiness     | Use locked phase-specific completion/approval/payment conditions                                            |
| Blocked gate              | Missing dependency/approval/payment/required artifact must be explicit                                      |

The Project Planning Agent defines/records operational readiness criteria, but actual phase transition authority remains with AgencyOS workflow/policy and responsible agents.

## 13. Planning State Machine

NOT\_STARTED → CONTEXT\_LOADING → ANALYZING\_SCOPE → BUILDING\_BLUEPRINT → CLARIFICATION\_REQUIRED / WAITING\_DEPENDENCY → VALIDATING\_PLAN → READY → VERSIONED\_UPDATE

A plan can be READY even though future client dependencies are not yet supplied, provided those dependencies are explicitly registered with needed-by points and they do not block the current kickoff gate.

## 14. Events & Handoffs

| Event                      | Producer        | Consumer              | Purpose                          |
|----------------------------|-----------------|-----------------------|----------------------------------|
| ProjectPlanningRequested   | PM/Orchestrator | Planning Agent        | Start operational planning       |
| PlanningContextLoaded      | Planning Agent  | Coordination          | Context validation evidence      |
| ClarificationRequired      | Planning Agent  | PM                    | Ask client without guessing      |
| ClarificationResolved      | PM              | Planning Agent        | Resume/update planning           |
| ClientDependencyIdentified | Planning Agent  | PM/Coordination       | Request/track early dependency   |
| PlanningBlockerIdentified  | Planning Agent  | Coordination/Admin/PM | Track blocker                    |
| ProjectPlanDrafted         | Planning Agent  | Validation workflow   | Validate structure/coverage      |
| ProjectPlanReady           | Planning Agent  | PM/Coordination       | Open pre-kickoff readiness check |
| ProjectPlanUpdated         | Planning Agent  | Affected workflows    | Versioned change notification    |

## 15. Minimum Data Model

| Entity               | Minimum fields                                                                               |
|----------------------|----------------------------------------------------------------------------------------------|
| ProjectPlan          | project_id, version, status, source refs, summary, assumptions, created/updated timestamps   |
| Deliverable          | id, plan_id, name, source, phase, owner, readiness, evidence, status                         |
| Dependency           | id, plan_id, type, description, owner, needed_by, status, blocker flag                       |
| OperationalMilestone | id, plan_id, phase/event, target window, dependencies, gate criteria, status                 |
| PlanningRisk         | id, plan_id, type, impact, likelihood/priority if configured, mitigation/owner, status       |
| Clarification        | id, plan_id, question, impact, PM/client response, source evidence, status                   |
| PlanChange           | plan/version, reason, source approval/clarification, before/after reference, actor/timestamp |
| AuditEvent           | actor, action, entity, version, evidence, timestamp                                          |

## 16. Permissions, Security & Governance

- ☐ Planning Agent receives only project-scoped context needed for planning.
- ☐ Sensitive credentials should be referenced as secure dependency records, not exposed in planning prose.
- ☐ Planning Agent cannot change Finance verification state.
- ☐ Planning Agent cannot change accepted quotation/scope without authorized workflow.
- ☐ Planning outputs are tenant/project isolated.
- ☐ Every plan version and material change is auditable.
- ☐ Client-facing communication is routed through PM.
- ☐ Internal AI provider/model details should not appear in client-facing planning artifacts.
- ☐ Historical plan versions remain recoverable for traceability.

## 17. Failure & Exception Handling

| Scenario                     | Required behavior                                                               |
|------------------------------|---------------------------------------------------------------------------------|
| Accepted scope missing       | Block planning; request valid source/handoff                                    |
| Conflicting requirements     | ClarificationRequired; no guessing                                              |
| Client dependency unknown    | Register as open dependency and route via PM                                    |
| Client dependency delayed    | Mark WAITING/BLOCKED at affected milestone; allow safe independent phases/tasks |
| External service uncertain   | Record dependency/assumption; do not invent availability                        |
| Timeline commitment conflict | Flag risk/clarification; preserve approved source                               |
| New requirement discovered   | Route to scope/change process                                                   |
| Duplicate planning trigger   | Return/reuse correct plan version idempotently                                  |
| Agent/job failure            | Retry from persisted state; avoid duplicate versions/events                     |
| Plan validation fails        | Return to BUILDING_BLUEPRINT with explicit validation errors                    |

## 18. Planning Validation Rules

- ☐ Every approved deliverable is represented or explicitly N/A with reason.
- ☐ Every deliverable has a source reference.
- ☐ Every applicable phase has an operational position.
- ☐ Dependencies have owners and needed-by points where possible.
- ☐ Client dependencies are visible early.
- ☐ Finance/approval gates are represented where applicable.
- ☐ No unresolved ambiguity is silently converted into scope.
- ☐ No development-level technical implementation plan is present.
- ☐ Plan schema validates.
- ☐ Plan is versioned and auditable.

# 19. Technical Implementation Plan - Planning Agent

## PLAN-I01 Repository Discovery

- ☐ Locate project models, quotation/scope data, Phase 1 handoff, Phase 2 onboarding, finance events, task/dependency system and phase state machine.
- ☐ Create requirement-to-code gap matrix.

## PLAN-I02 Structured Planning Schema

- ☐ Implement ProjectPlan, Deliverable, Dependency, OperationalMilestone, PlanningRisk, Clarification and PlanChange.
- ☐ Add versioning and audit.

## PLAN-I03 Context Loader

- ☐ Load accepted scope/quote/requirements/onboarding context.
- ☐ Validate source completeness and prevent cross-project leakage.

## PLAN-I04 Scope Parser / Deliverables Builder

- ☐ Convert approved scope into structured deliverables without adding features.
- ☐ Attach source references.

## PLAN-I05 Phase & Milestone Mapper

- ☐ Map deliverables to AgencyOS phases.
- ☐ Map operational milestones, finance gates and approvals.

## PLAN-I06 Dependency Engine

- ☐ Create client/internal/external/finance/approval dependencies.
- ☐ Support owner, needed-by, status and blocker relationships.

## PLAN-I07 Timeline Shell

- ☐ Generate operational sequence/windows from approved commitments and dependencies.
- ☐ Persist assumptions.

## PLAN-I08 Clarification Loop

- ☐ Emit ClarificationRequired.
- ☐ Integrate PM/client structured response.
- ☐ Version plan after resolution.

## PLAN-I09 Readiness & Validation

- ☐ Validate coverage, dependencies, ambiguity and role boundary.
- ☐ Emit ProjectPlanReady only after validation pass.

## PLAN-I10 Observability & Recovery

- ☐ Add traces, audit, retry/idempotency, blocker visibility and version history.

## PLAN-I11 QA

- ☐ Unit-test parsers/state/validation.
- ☐ Integration-test PM/Finance/phase events.
- ☐ E2E-test happy path, ambiguity, delayed dependencies and plan update.

## 20. QA Test Matrix

| Area             | Required tests                                             |
|------------------|------------------------------------------------------------|
| Trigger          | Valid request; duplicate request; invalid project          |
| Context          | Complete; missing scope; cross-project isolation           |
| Deliverables     | All approved deliverables mapped; no invented deliverables |
| Phases           | Correct applicable sequence; N/A handling                  |
| Dependencies     | Client/internal/external/approval/finance; blockers        |
| Timeline         | Approved commitment; assumptions; delayed dependency       |
| Clarification    | Ambiguity → PM → response → version update                 |
| Scope protection | New requirement not silently added                         |
| Role boundary    | No code/DB/API implementation plan generated               |
| Versioning       | Draft/update/history/source attribution                    |
| Readiness        | Validation pass/fail and missing-gate reporting            |
| Recovery         | Retry, replay, duplicate trigger, persisted state          |
| Security         | Tenant/project isolation; sensitive dependency handling    |
| Audit            | Every material plan change attributable                    |

## 21. Definition of Done

- ☐ Project Planning Agent starts from valid Phase 2 context.
- ☐ Accepted scope is converted into a complete deliverables register.
- ☐ Applicable AgencyOS phases and operational sequence are mapped.
- ☐ Client dependencies are identified early.
- ☐ Internal/external/approval/finance dependencies are represented.
- ☐ Operational timeline shell and assumptions are stored.
- ☐ Risk/blocker register is created.
- ☐ Clarification loop works through PM without guessing.
- ☐ New/out-of-scope requirements route correctly.
- ☐ Readiness criteria/evidence expectations are defined.
- ☐ Planning output is versioned and auditable.
- ☐ ProjectPlanReady event is emitted after validation.
- ☐ Agent does not perform Phase 5 development planning.
- ☐ Permissions, isolation, idempotency and recovery tests pass.
- ☐ No unresolved P0/P1 planning workflow defects remain.

## 22. Master Project Planning Checklist

### A. Context

- ☐ Project/quote linked.
- ☐ Approved scope loaded.
- ☐ Requirements loaded.
- ☐ Onboarding context loaded.
- ☐ Payment/planning trigger validated.

## B. Deliverables

- ☐ Deliverables extracted.
- ☐ Source refs.
- ☐ Phase mapping.
- ☐ Owner roles.
- ☐ Readiness criteria.
- ☐ Evidence.

## C. Dependencies

- ☐ Client dependencies.
- ☐ Internal dependencies.
- ☐ External services.
- ☐ Approvals.
- ☐ Finance gates.
- ☐ Needed-by points.
- ☐ Blocker status.

## D. Timeline

- ☐ Phase sequence.
- ☐ Milestones.
- ☐ Target windows.
- ☐ Assumptions.
- ☐ Parallel-safe work.
- ☐ Delay handling.

## E. Clarifications

- ☐ Ambiguity detection.
- ☐ PM handoff.
- ☐ Client response evidence.
- ☐ Scope-change detection.
- ☐ Plan update/version.

## F. Risks & Readiness

- ☐ Risk register.
- ☐ Blockers.
- ☐ Readiness gates.
- ☐ Validation.
- ☐ ProjectPlanReady.

## G. Governance

- ☐ No technical overreach.
- ☐ No scope invention.
- ☐ Versioning.
- ☐ Audit.
- ☐ Permissions.
- ☐ Isolation.

## H. Quality

- ☐ Unit tests.

- ☐ Integration tests.
- ☐ E2E.
- ☐ Negative tests.
- ☐ Idempotency.
- ☐ Retry/recovery.
- ☐ Validation tests.

## 23. Claude Code Execution Protocol

DISCOVER EXISTING PLANNING/PROJECT INFRASTRUCTURE → MAP THIS PDF → IDENTIFY GAPS → IMPLEMENT STRUCTURED PLAN DOMAIN → TEST → IMPLEMENT DEPENDENCIES/TIMELINE/CLARIFICATION → TEST → VALIDATE ROLE BOUNDARY → FULL PHASE 2 E2E → PRODUCTION READINESS

- ☐ Do not rewrite working AgencyOS foundations unnecessarily.
- ☐ Use existing project/task/event/audit conventions when equivalent.
- ☐ Do not merge Project Planning Agent with Development Planning Agent.
- ☐ Do not invent client requirements.
- ☐ Do not silently modify accepted scope.
- ☐ Do not directly ask the client when PM owns communication.
- ☐ Do not store secrets in generated planning prose.
- ☐ Every checklist item requires implementation evidence/tests before completion.
- ☐ Final report must include changed files, migrations, tests, manual configuration, limitations and production-readiness status.

## 24. Final Locked Project Planning Flow

PHASE 2 CONTEXT + VERIFIED FINANCIAL GATE → PROJECT PLANNING REQUESTED → LOAD ACCEPTED SCOPE/REQUIREMENTS → BUILD DELIVERABLES REGISTER → MAP AGENCYOS PHASES/MILESTONES → IDENTIFY CLIENT + INTERNAL + EXTERNAL + APPROVAL + FINANCE DEPENDENCIES → CREATE TIMELINE SHELL → CREATE RISKS/BLOCKERS → CLARIFICATION LOOP THROUGH PM IF NEEDED → DEFINE READINESS/EVIDENCE → VALIDATE → VERSION PROJECT BLUEPRINT → PROJECT PLAN READY → PM PRE-KICKOFF READINESS → OFFICIAL KICKOFF

Locked principle: Project Planning Agent answers “How will this project be organized and move through AgencyOS?” The later Development Planning Agent answers “How will the product technically be built?” Keep these responsibilities strictly separate.